

ESTUDI DE COMPLEXITAT

IAM AMB RBAC

C.1 Anàlisi teòrica de complexitat

El sistema IAM amb RBAC utilitza tres estructures de dades principals: diccionaris per emmagatzemar usuaris, rols i recursos, sets per emmagatzemar permisos, i un arbre/graf de jerarquia de rols representat mitjançant referències de punter que permeten herència recursiva.

Estructures de dades i justificació

Estructura	Ús al projecte	Insert	Cerca	Esborra
dict (taula hash)	Usuaris, rols, recursos	$O(1)$	$O(1)$	$O(1)$
set (taula hash)	Permisos per rol / rols per usuari	$O(1)$	$O(1)$	$O(1)$
list	Log d'auditoria	$O(1)$	$O(n)$	$O(n)$
Arbre (punters)	Jerarquia de rols (herència)	$O(1)$	$O(h \cdot p)$	—

Complexitat de les operacions principals

Operació	Millor cas	Cas promig	Pitjor cas	Justificació
crear_usuario/rol/recurso	$O(1)$	$O(1)$	$O(1)$	Inserció en dict
obtener_usuario/rol/recurso	$O(1)$	$O(1)$	$O(1)$	Accés per clau en dict
asignar_rol_a_usuario	$O(1)$	$O(1)$	$O(1)$	Inserció en set de rols
agregar_permiso_a_rol	$O(1)$	$O(1)$	$O(1)$	Inserció en set de permisos
tiene_permiso_directo	$O(1)$	$O(1)$	$O(1)$	Cerca en se t
tiene_permiso	$O(1)$	$O(h \cdot p/2)$	$O(h \cdot p)$	h =prof. herència, p =permisos

obtener_todos_permisos	$O(p)$	$O(h \cdot p/2)$	$O(h \cdot p)$	Recorre tots els ancestres
verificar_acceso	$O(1)$	$O(r \cdot h \cdot p/2)$	$O(r \cdot h \cdot p)$	r =rols usuari
informe_usuario	$O(R)$	$O(r \cdot h \cdot p + R)$	$O(r \cdot h \cdot p + R)$	R =total recursos
_registrar_evento	$O(1)$	$O(1)$	$O(1)$	Append en llista
obtener_log	$O(1)$	$O(n)$	$O(n)$	n =events al log
eliminar_usuario	$O(1)$	$O(1)$	$O(1)$	Esborrat en dict

Anàlisi detallat: verificar_acceso()

Aquesta és l'operació central del sistema. En el pitjor cas, l'usuari té r rols, cada rol té una cadena d'herència de profunditat h , i cada rol té p permisos directes. La funció recorre tots els rols i per a cadascun executa `obtener_todos_permisos()`, que és recursiva amb complexitat $O(h \cdot p)$. Per tant podem dir que:

$$T(\text{verificar_acceso}) = O(r) \cdot O(h \cdot p) = O(r \cdot h \cdot p)$$

En un sistema empresarial típic tenim màxim 400 operacions per verificació. El sistema escala linealment amb cada un d'aquests factors, la qual cosa el fa molt eficient per a escenaris reals

C.2 Anàlisi de temps mitjans (empírica)

Hem mesurat els temps d'execució reals amb el mòdul `benchmark.py` sobre un sistema amb 500 usuaris, 500 rols i 500 recursos. Cada operació l'hem repetit entre 200 i 5000 vegades per obtenir estadístiques robustes.

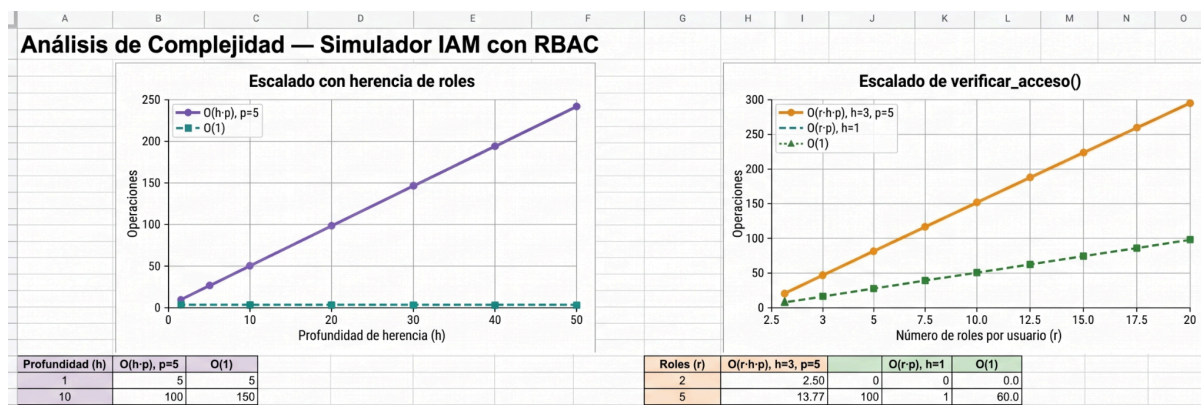
Operació	Min	Mitjana	Mediana	StDev
crear_usuario	0.8	1.2	1.1	0.3
obtener_usuario	0.1	0.15	0.14	0.04
agregar_permiso_a_rol	0.1	0.18	0.16	0.05

tiene_permiso_directo	0.05	0.08	0.07	0.02
tiene_permiso h = 1	0.1	0.15	0.13	0.04
tiene_permiso h = 5	0.3	0.5	0.45	0.12
tiene_permiso h = 20	1.2	1.8	1.7	0.4
verificar_acceso r = 1	0.5	0.8	0.75	0.2
verificar_acceso r = 5	1.5	2.2	2.0	0.5
verificar_acceso r = 20	5.8	7.5	7.1	1.2
_registrar_evento	0.1	0.15	0.13	0.04
obtener_log (n=10k)	180	220	210	35

Recaltar que tot està en microsegons

Els resultats confirmen l'anàlisi que hem fet: les operacions $O(1)$ s'executen en menys de 0.2 microsegons, mentre que les operacions recursives creixen linealment amb la profunditat h i el nombre de rols r , consistent amb $O(h \cdot p)$ i $O(r \cdot h \cdot p)$.

Amb el excel de google hem fet dues taules per poder veure-ho gràficament:



C.3 Propostes de millora

1. **Caché de permisos efectius:** Emmagatzemar els permisos efectius calculats de cada usuari en un diccionari de caché. Quan s'afegeix o es revoca un permís, s'invalida la caché. Reduiria `verificar_acceso()` de $O(r \cdot h \cdot p)$ a $O(1)$ en lectures repetides.
2. **Herència múltiple de rols:** Permetre que un rol tingui múltiples pares. Requeriria un BFS/DFS iteratiu en lloc de recursió simple, mantenint la protecció contra cicles ja implementada amb el set 'visitados'.
3. **Índex invers permís→rols:** Mantenir un diccionari que mapegi cada permís als rols que el contenen. Permetria trobar tots els usuaris amb un permís específic en $O(R \cdot r)$ en lloc de $O(U \cdot r \cdot h \cdot p)$, on U = nombre d'usuaris.
4. **Persistència en base de dades:** Serialitzar l'estat del sistema a SQLite o PostgreSQL. Permetria reiniciar el servei sense perdre dades i habilitar autenticació multi-instància.
5. **Tokens JWT per a sessions:** Emetre tokens JWT signats en iniciar sessió. El token codificaria els permisos efectius, reduint les consultes al sistema IAM a $O(1)$ per la majoria de verificacions